

Cheat Sheet – Regression Analysis

What is Regression Analysis?

Fitting a function $f(\cdot)$ to datapoints $y_i=f(x_i)$ under some error function. Based on the estimated function and error, we have the following types of regression

1. Linear Regression:

Fits a **line** minimizing the **sum of mean-squared error** for each datapoint.

$$\min_{\beta} \sum_i \|y_i - f_{\beta}^{\text{linear}}(x_i)\|^2$$

$$f_{\beta}^{\text{linear}}(x_i) = \beta_0 + \beta_1 x_i$$

2. Polynomial Regression:

Fits a **polynomial** of order k ($k+1$ unknowns) minimizing the **sum of mean-squared error** for each datapoint.

$$\min_{\beta} \sum_{i=0}^m \|y_i - f_{\beta}^{\text{poly}}(x_i)\|^2$$

$$f_{\beta}^{\text{poly}}(x_i) = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \dots + \beta_k x_i^k$$

3. Bayesian Regression:

For each datapoint, fits a **gaussian distribution** by minimizing the **mean-squared error**. As the number of data points x_i increases, it converges to point estimates i.e. $n \rightarrow \infty, \sigma^2 \rightarrow 0$

$$\min_{\beta} \sum_i \|y_i - \mathcal{N}(f_{\beta}(x_i), \sigma^2)\|^2$$

$$f_{\beta}(x_i) = f_{\beta}^{\text{poly}}(x_i) \text{ or } f_{\beta}^{\text{linear}}(x_i)$$

$$\mathcal{N}(\mu, \sigma^2) \rightarrow \text{Gaussian with mean } \mu \text{ and variance } \sigma^2$$

4. Ridge Regression:

Can fit either a **line**, or **polynomial** minimizing the **sum of mean-squared error** for each datapoint and the **weighted L2 norm** of the function parameters beta.

$$\min_{\beta} \sum_{i=0}^m \|y_i - f_{\beta}(x_i)\|^2 + \sum_{j=0}^k \beta_j^2$$

$$f_{\beta}(x_i) = f_{\beta}^{\text{poly}}(x_i) \text{ or } f_{\beta}^{\text{linear}}(x_i)$$

5. LASSO Regression:

Can fit either a **line**, or **polynomial** minimizing the **sum of mean-squared error** for each datapoint and the **weighted L1 norm** of the function parameters beta.

$$\min_{\beta} \sum_{i=0}^m \|y_i - f_{\beta}(x_i)\|^2 + \sum_{j=0}^k |\beta_j|$$

$$f_{\beta}(x_i) = f_{\beta}^{\text{poly}}(x_i) \text{ or } f_{\beta}^{\text{linear}}(x_i)$$

6. Logistic Regression:

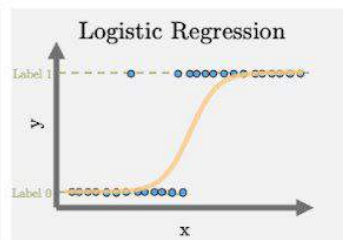
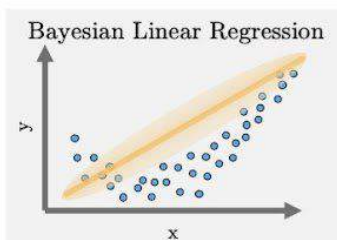
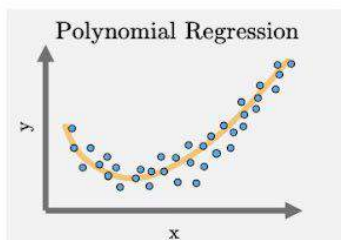
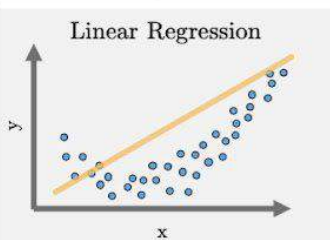
Can fit either a **line**, or **polynomial with sigmoid activation** minimizing the **binary cross-entropy loss** for each datapoint. The labels y are binary class labels.

$$\min_{\beta} \sum_i -y_i \log(\sigma(f_{\beta}(x_i))) - (1 - y_i) \log(1 - \sigma(f_{\beta}(x_i)))$$

$$f_{\beta}(x_i) = f_{\beta}^{\text{poly}}(x_i) \text{ or } f_{\beta}^{\text{linear}}(x_i)$$

$$\sigma(t) = \frac{1}{1 + e^{-t}}$$

Visual Representation:



Summary:

	What does it fit?	Estimated function	Error Function
Linear	A line in n dimensions	$f_{\beta}^{\text{linear}}(x_i) = \beta_0 + \beta_1 x_i$	$\sum_{i=0}^m \ y_i - f_{\beta}(x_i)\ ^2$
Polynomial	A polynomial of order k	$f_{\beta}^{\text{poly}}(x_i) = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \dots$	$\sum_{i=0}^m \ y_i - f_{\beta}(x_i)\ ^2$
Bayesian Linear	Gaussian distribution for each point	$\mathcal{N}(f_{\beta}(x_i), \sigma^2)$	$\sum_i \ y_i - \mathcal{N}(f_{\beta}(x_i), \sigma^2)\ ^2$
Ridge	Linear/polynomial	$f_{\beta}^{\text{poly}}(x_i) \text{ or } f_{\beta}^{\text{linear}}(x_i)$	$\sum_{i=0}^m \ y_i - f_{\beta}(x_i)\ ^2 + \sum_{j=0}^n \beta_j^2$
LASSO	Linear/polynomial	$f_{\beta}^{\text{poly}}(x_i) \text{ or } f_{\beta}^{\text{linear}}(x_i)$	$\sum_{i=0}^m \ y_i - f_{\beta}(x_i)\ ^2 + \sum_{j=0}^n \beta_j $
Logistic	Linear/polynomial with sigmoid	$\sigma(f_{\beta}(x_i))$	$\min_{\beta} \sum_i -y_i \log(\sigma(f_{\beta}(x_i))) - (1 - y_i) \log(1 - \sigma(f_{\beta}(x_i)))$

Cheat Sheet – Convolutional Neural Network

Convolutional Neural Network:

The data gets into the CNN through the input layer and passes through various hidden layers before getting to the output layer. The output of the network is compared to the actual labels in terms of loss or error. The partial derivatives of this loss w.r.t the trainable weights are calculated, and the weights are updated through one of the various methods using backpropagation.

CNN Template:

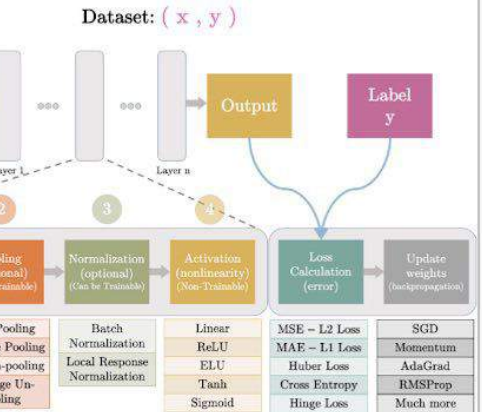
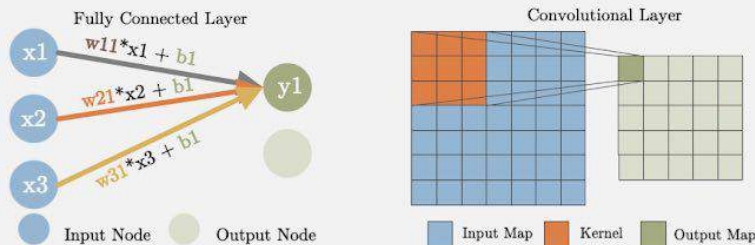
Most of the commonly used hidden layers (not all) follow a pattern

1. **Layer function:** Basic transforming function such as convolutional or fully connected layer.

a. **Fully Connected:** Linear functions between the input and the

a. **Convolutional Layers:** These layers are applied to 2D (3D) input feature maps. The trainable weights are a 2D (3D) kernel/filter that moves across the input feature map, generating dot products with the overlapping region of the input feature map.

b. **Transposed Convolutional (DeConvolutional) Layer:** Usually used to increase the size of the output feature map (Upsampling) The idea behind the transposed convolutional layer is to undo (not exactly) the convolutional layer



Convolutional	Max Pooling	Batch Normalization	Linear	MSE - L2 Loss	SGD
Transposed Convolutional	Average Pooling	Local Response Normalization	ReLU	MAE - L1 Loss	Momentum
Fully Connected	Max Un-pooling	Average Un-pooling	ELU	Huber Loss	AdaGrad
			Tanh	Cross Entropy	RMSProp
			Sigmoid	Hinge Loss	Much more

2. **Pooling:** Non-trainable layer to change the size of the feature map

a. **Max/Average Pooling:** Decrease the spatial size of the input layer based on selecting the maximum/average value in receptive field defined by the kernel

b. **UnPooling:** A non-trainable layer used to increase the spatial size of the input layer based on placing the input pixel at a certain index in the receptive field of the output defined by the kernel.

3. **Normalization:** Usually used just before the activation functions to limit the unbounded activation from increasing the output layer values too high

a. **Local Response Normalization LRN:** A non-trainable layer that square-normalizes the pixel values in a feature map within a local neighborhood.

b. **Batch Normalization:** A trainable approach to normalizing the data by learning scale and shift variable during training.

3. **Activation:** Introduce non-linearity so CNN can efficiently map non-linear complex mapping.

a. **Non-parametric/Static functions:** Linear, ReLU

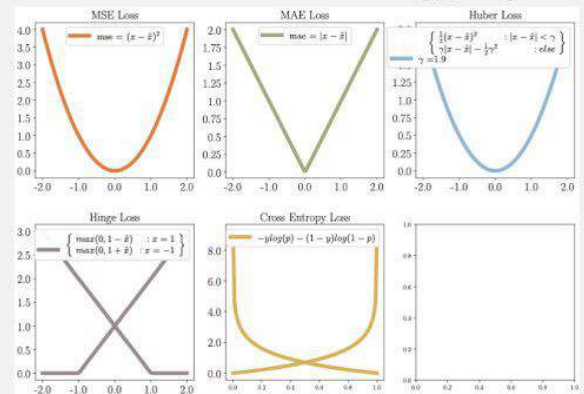
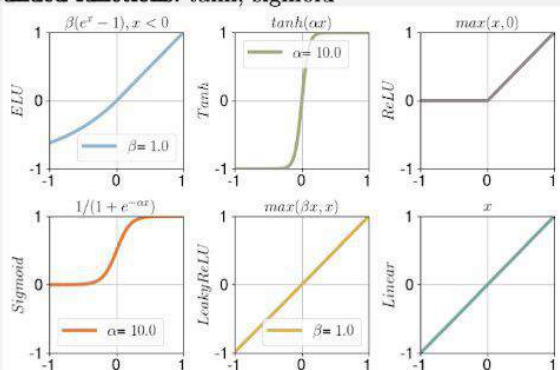
b. **Parametric functions:** ELU, tanh, sigmoid, Leaky ReLU

c. **Bounded functions:** tanh, sigmoid

5. **Loss function:** Quantifies how far off the CNN prediction is from the actual labels.

a. **Regression Loss Functions:** MAE, MSE, Huber loss

b. **Classification Loss Functions:** Cross entropy, Hinge loss



Cheat Sheet – Bayes Theorem and Classifier

What is Bayes' Theorem?

- Describes the probability of an event, based on prior knowledge of conditions that might be related to the event.

$$P(A|B) = \frac{P(B|A)(\text{likelihood}) \times P(A)(\text{prior})}{P(B)(\text{evidence})}$$

- How the probability of an event changes when we have knowledge of another event

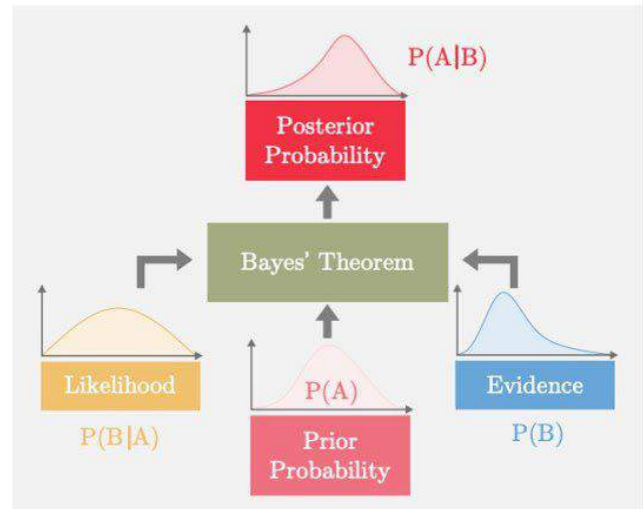
$$P(A) \longrightarrow P(A|B)$$

Usually, a better estimate than $P(A)$

Example

- Probability of fire $P(F) = 1\%$
- Probability of smoke $P(S) = 10\%$
- Prob of smoke given there is a fire $P(S|F) = 90\%$
- What is the probability that there is a fire given we see a smoke $P(F|S)$?

$$P(F|S) = \frac{P(S|F) \times P(F)}{P(S)} = \frac{0.9 \times 0.01}{0.1} = 9\%$$



Maximum A Posteriori Probability (MAP) Estimation

The MAP estimate of the random variable y , given that we have observed iid $(x_1, x_2, x_3, \dots)$, is given by. We try to accommodate our prior knowledge when estimating.

$$\hat{y}_{MAP} = \underset{y}{\operatorname{argmax}} P(y) \prod_i P(x_i|y)$$

y that maximizes the product of **prior** and **likelihood**

Maximum Likelihood Estimation (MLE)

The MAP estimate of the random variable y , given that we have observed iid $(x_1, x_2, x_3, \dots)$, is given by. We assume we don't have any prior knowledge of the quantity being estimated.

$$\hat{y}_{MLE} = \underset{y}{\operatorname{argmax}} \prod_i P(x_i|y)$$

y that maximizes only the **likelihood**

MLE is a special case of MAP where our prior is uniform (all values are equally likely)

Naïve Bayes' Classifier (Instantiation of MAP as classifier)

Suppose we have two classes, $y=y_1$ and $y=y_2$. Say we have more than one evidence/features $(x_1, x_2, x_3, \dots)$, using Bayes' theorem

$$P(y|x_1, x_2, x_3, \dots) = \frac{P(x_1, x_2, x_3, \dots|y) \times P(y)}{P(x_1, x_2, x_3, \dots)}$$

Bayes' theorem assumes the features $(x_1, x_2, x_3, \dots)$ are i.i.d. i.e $P(x_1, x_2, x_3, \dots|y) = \prod_i P(x_i|y)$

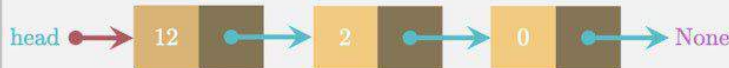
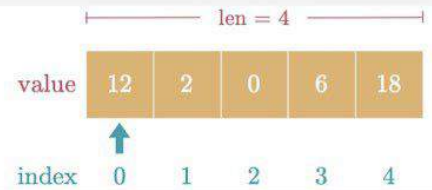
$$P(y|x_1, x_2, x_3, \dots) = \prod_i P(x_i|y) \frac{P(y)}{P(x_1, x_2, x_3, \dots)}$$

$$\hat{y} = y_1 \text{ if } \frac{P(y_1|x_1, x_2, x_3, \dots)}{P(y_2|x_1, x_2, x_3, \dots)} > 1 \text{ else } \hat{y} = y_2$$

Cheat Sheet – Data Structures

1. List

- Ordered collection of elements
- The position of each element is defined by the *index*
- The elements can be accessed in *any order*



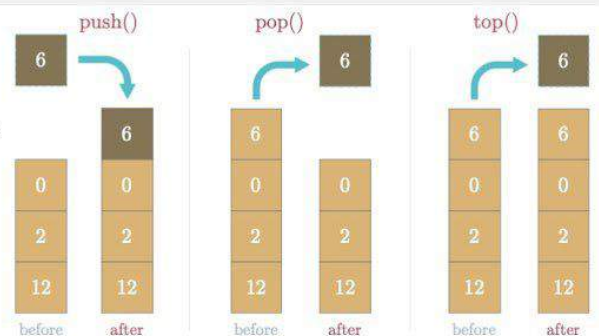
- Each linked list element contains both the values and the address (pointer) to the next linked list element.
- Hence the linked list can only be traversed sequentially going through each element at a time.

2. Linked List

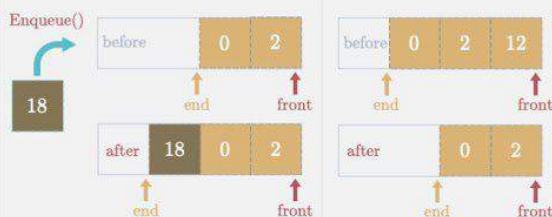
- Linked List does not have their order defined by their physical placement in the memory.
- Contiguous elements of the linked list are not placed adjacent to each other in the memory.

3. Stack

- Stack is a sequential data structure which maintains the order of elements as they were inserted in.
- Last In First Out (LIFO) order, which means that the elements can only be accessed in the reverse order as they were inserted into the stack.
- The element to be inserted last, will be the first one to get removed from the stack.
- Push() adds an element at the head of the stack, while pop() removes an element from the head of the stack
- A real-life example of a stack is a stack of kitchen plates



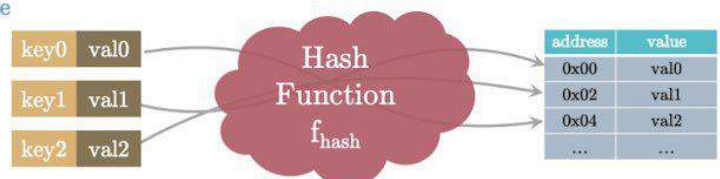
4. Queue



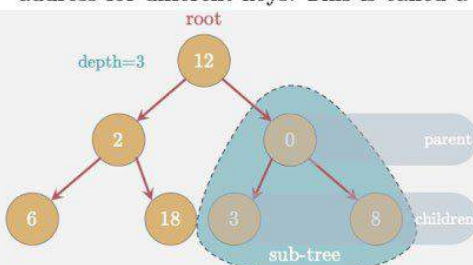
- A queue is a sequential data structure that maintains the order of elements as they were inserted in
- First In First Out (FIFO), the element to be inserted first, will be the first one to get removed from the queue
- Whenever an element is added (Enqueue()) it is added to the end of the queue. On the other hand, element removal (Dequeue()) is done from the front of the queue.
- A real-life example is a check-out line at a grocery store

5. HashTable

- Creates paired assignments (key mapped to values) so the pairs can be accessed in constant time
- For each (key, value) pair, the key is passed through a hash function to create a unique physical address for the value to be stored in the memory.
- Hash function can end up generating the same physical address for different keys. This is called a collision.



4. Tree



- Maintains a hierarchical relation between its elements.
- Root Node – The node at the top of the tree
- Parent Node – Any node that has at least one child
- Child Node – The successor of a parent node is known as a child node. A node can be both a parent and a child node. The root is never a child node.
- Leaf Node – The node which does not have any child node.
- Traversing – Passing through the nodes in a certain order, e.g BFS, DFS

4. Graph

- A graph is a pair of sets (V, E), where V is set of all the vertices, E is set of all edges.
- A neighbor of a node is set of all vertices connected with that node through an edge.
- As opposed to trees, a graph can be cyclic, which means starting from a node and following the edges, you can end up on the same node

